

```

# Load the libraries we need to use
library(PSG)
library(dplyr)
library(tidyr)
library(xts)
library(ggplot2)

setwd("C:/Users/JUANJO/Documents/GitHub/ams-518/homework-7/data")

# Set path to Rdata file including the RData file's name with CS:
load("C:/Users/JUANJO/Documents/GitHub/ams-518/homework-7/data/problem_st_dev_risk.RData")

load("C:/Users/JUANJO/Documents/GitHub/ams-518/homework-7/data/problem_CVaR2_err_075.RData")

# Check what is the current problem statement
problem.list$problem_statement

# Modify the problem statement
problem.list$problem_statement[1] <- "for {matrix_fact_in; matrix_fact_out; num} =
crossvalidation(5, matrix_scenarios)"
problem.list$problem_statement[2] <- "minimize"
problem.list$problem_statement[3] <- "cvar_dev(0.95,matrix_fact_in)"
problem.list$problem_statement[4] <- "Value:"
problem.list$problem_statement[5] <- "L_num(matrix_fact_out)"
problem.list$problem_statement[6] <- "L_num(matrix_fact_in)"
problem.list$problem_statement[7] <- "cvar_dev(0.95, matrix_fact_in)"
problem.list$problem_statement[8] <- "meanabs_dev(matrix_fact_in)"
problem.list$problem_statement[9] <- "st_dev(matrix_fact_in)"
problem.list$problem_statement[10] <- "var_risk(0.75, matrix_fact_in)"
problem.list$problem_statement[11] <- "var_risk_g(0.75, matrix_fact_in)"
problem.list$problem_statement[12] <- "var_risk(0.9, matrix_fact_in)"
problem.list$problem_statement[13] <- "var_risk_g(0.9, matrix_fact_in)"
problem.list$problem_statement[14] <- "end for"

# Save the problem statement for future iterations
problem_statement <- problem.list$problem_statement

# Extract the original 3-factor matrix (e.g., 1000 rows, 4 cols)
matrix_scenarios_3fact <- problem.list$matrix_scenarios

# Last name: Perez, hence I should pick variables x2 and x3
matrix_scenarios_2fact <- matrix_scenarios_3fact[, c(2, 3, 4)]

# Set up our simulation parameters

# Create a sequence of training sizes of 10, 20, 40, 60, ..., 900
training_sizes <- c(10, seq(20, 900, by = 20))

# Number of times we are going to run each simulation (minimum = 20)
n_repetitions <- 30

# Total number of rows
n_total_rows <- nrow(matrix_scenarios_3fact)

# Testing set will always contain 100 rows
n_test <- 100

all_indices <- 1:n_total_rows

# Initialize the results storage
# This data frame will store the final average for each training size
results_df <- data.frame(
  TrainingSize = integer(),

```

```

# 2-Factor In-Sample (x2, x3)
Avg_InSample_2F_f1 = double(), # x2
Avg_InSample_2F_f2 = double(), # x3

# 2-Factor Out-of-Sample
Avg_OutOfSample_2F_f1 = double(), # x2
Avg_OutOfSample_2F_f2 = double(), # x3

# 3-Factor In-Sample (x1, x2, x3)
Avg_InSample_3F_f1 = double(), # x1
Avg_InSample_3F_f2 = double(), # x2
Avg_InSample_3F_f3 = double(), # x3

# 3-Factor Out-of-Sample
Avg_OutOfSample_3F_f1 = double(), # x1
Avg_OutOfSample_3F_f2 = double(), # x2
Avg_OutOfSample_3F_f3 = double() # x3
)

# Initialize the problem lists
problem.list.in.2f <- problem.list # Use the existing problem.list
problem.list.in.3f <- problem.list # Use the existing problem.list

# Outer loop: to iterate over each training size
for (n_train in training_sizes) {

  print(paste("--- Starting Training Size:", n_train, "---"))

  # Temporary matrices to store the average vectors from each rep
  # Matrix: (n_repetitions x 2 factors)
  in_sample_2f_reps <- matrix(NA, nrow = n_repetitions, ncol = 2)
  # Matrix: (n_repetitions x 3 factors)
  in_sample_3f_reps <- matrix(NA, nrow = n_repetitions, ncol = 3)

  # Temporary vectors to store results for the 30 repetitions
  out_sample_2f_reps <- matrix(NA, nrow = n_repetitions, ncol = 2)
  out_sample_3f_reps <- matrix(NA, nrow = n_repetitions, ncol = 3)

  # Repeat the experiment 30 times
  for (i in 1:n_repetitions) {

    # Step I: Randomly select 100 rows for testing set
    test_indices <- sample(all_indices, n_test)
    remaining_indices <- setdiff(all_indices, test_indices) # 900 remaining

    # Step II: Randomly select n_train rows from remaining
    train_indices <- sample(remaining_indices, n_train)

    # Create the training and testing matrices for this iteration
    matrix_train_set_3f <- matrix_scenarios_3fact[train_indices, ]
    matrix_test_set_3f <- matrix_scenarios_3fact[test_indices, ]

    matrix_train_set_2f <- matrix_scenarios_2fact[train_indices, ]
    matrix_test_set_2f <- matrix_scenarios_2fact[test_indices, ]

    #Step III: Fit models and get In-Sample CVaR Deviation

    # 2-Factor In-Sample

    # Modify the first entry of the statement
    problem.list.in.2f$problem_statement[1] <-
      "for {matrix_fact_in; matrix_fact_out; num} = crossvalidation(5, matrix_train_set_2f)"
    # Modify the training set
    problem.list.in.2f$matrix_scenarios <- matrix_train_set_2f

```

```

# Generate output for 2 factor model
output_in_2f <- rpsg_solver(problem.list.in.2f)

# Extract the CVaR value from each of the 5 list items and average them
cvar_vals_2f <- lapply(output_in_2f, function(x) {
  # Find the element that starts with "point_problem_"
  point_name <- grep("^point_problem_", names(x), value = TRUE)
  if(length(point_name) > 0) {
    return(x[[point_name]])
  } else {
    return(NULL)
  }
})

# Extract x2 values and average
avg_x2_2f <- mean(sapply(cvar_vals_2f, `[`, "x2"))
# Extract x3 values and average
avg_x3_2f <- mean(sapply(cvar_vals_2f, `[`, "x3"))

# Fill in the averaged values
in_sample_2f_reps[i,] <- c(avg_x2_2f, avg_x3_2f)

# 3-Factor In-Sample

# Modify the first entry of the statement
problem.list.in.3f$problem_statement[1] <-
  "for {matrix_fact_in; matrix_fact_out; num} = crossvalidation(5, matrix_train_set_3f)"
# Modify the training set
problem.list.in.3f$matrix_scenarios <- matrix_train_set_3f
output_in_3f <- rpsg_solver(problem.list.in.3f)

# Extract the CVaR value from each of the 5 list items and average them
cvar_vals_3f <- lapply(output_in_3f, function(x) {
  # Find the element that starts with "point_problem_"
  point_name <- grep("^point_problem_", names(x), value = TRUE)
  if(length(point_name) > 0) {
    return(x[[point_name]])
  } else {
    return(NULL)
  }
})

# Extract x1 values and average
avg_x1_3f <- mean(sapply(cvar_vals_3f, `[`, "x1"))
# Extract x2 values and average
avg_x2_3f <- mean(sapply(cvar_vals_3f, `[`, "x2"))
# Extract x3 values and average
avg_x3_3f <- mean(sapply(cvar_vals_3f, `[`, "x3"))

# Fill in the averaged values
in_sample_3f_reps[i,] <- c(avg_x1_3f, avg_x2_3f, avg_x3_3f)

# Step IV: Calculate Out-Of-Sample CVaR Deviation

# 2-Factor Out-of-Sample

problem.list.out.2f <- list()
problem.list.out.2f$problem_statement[1] <- "minimize"
problem.list.out.2f$problem_statement[2] <- "cvar_dev(0.95, matrix_s)"

# Get the number of rows from the test set
n_rows_2f <- nrow(matrix_test_set_2f)
# Create the intercept column (1s)

```

```

intercept <- rep(1, n_rows_2f)

# Build the new matrix in the required order:
# [intercept, scenario_benchmark, x2, x3]
matrix_s_2f <- cbind(
  intercept = intercept,
  scenario_benchmark = matrix_test_set_2f[, 3], # 3rd col (scenario_benchmark)
  x2 = matrix_test_set_2f[, 1],                # 1st col (x2)
  x3 = matrix_test_set_2f[, 2]                # 2nd col (x3)
)

problem.list.out.2f$matrix_s <- matrix_s_2f
output_out_2f <- rpsg_solver(problem.list.out.2f)

# Extract x2 value
x2_2f <- output_out_2f$point_problem_1["x2"]
# Extract x3 value
x3_2f <- output_out_2f$point_problem_1["x3"]

out_sample_2f_reps[i,] <- c(x2_2f, x3_2f)

# 3-Factor Out-of-Sample
problem.list.out.3f <- list()
problem.list.out.3f$problem_statement[1] <- "minimize"
problem.list.out.3f$problem_statement[2] <- "cvar_dev(0.95, matrix_s)"

# Get the number of rows from the test set
n_rows_3f <- nrow(matrix_test_set_3f)
# Create the intercept column (1s)
intercept <- rep(1, n_rows_3f)

# Build the new matrix in the required order:
# [intercept, scenario_benchmark, x1, x2, x3]
matrix_s_3f <- cbind(
  intercept = intercept,
  scenario_benchmark = matrix_test_set_3f[, 4], # 4th col (scenario_benchmark)
  x1 = matrix_test_set_3f[, 1],                # 1st col (x1)
  x2 = matrix_test_set_3f[, 2],                # 2nd col (x2)
  x3 = matrix_test_set_3f[, 3]                # 3rd col (x3)
)

problem.list.out.3f$matrix_s <- matrix_s_3f
output_out_3f <- rpsg_solver(problem.list.out.3f)

# Extract x1 value
x1_3f <- output_out_3f$point_problem_1["x1"]
# Extract x2 value
x2_3f <- output_out_3f$point_problem_1["x2"]
# Extract x3 value
x3_3f <- output_out_3f$point_problem_1["x3"]

out_sample_3f_reps[i,] <- c(x1_3f, x2_3f, x3_3f)

} # End of middle loop (n_repetitions)

# Step V: Calculate averages
avg_in_2f <- colMeans(in_sample_2f_reps, na.rm = TRUE)
avg_in_3f <- colMeans(in_sample_3f_reps, na.rm = TRUE)

avg_out_2f <- colMeans(out_sample_2f_reps, na.rm = TRUE)
avg_out_3f <- colMeans(out_sample_3f_reps, na.rm = TRUE)

# Store the averages in the main results data frame
results_df <- rbind(results_df, data.frame(
  TrainingSize = n_train,

```

```

Avg_InSample_2F_f1 = avg_in_2f[1], # Corresponds to x2
Avg_InSample_2F_f2 = avg_in_2f[2], # Corresponds to x3

Avg_OutOfSample_2F_f1 = avg_out_2f[1], # Corresponds to x2
Avg_OutOfSample_2F_f2 = avg_out_2f[2], # Corresponds to x3

Avg_InSample_3F_f1 = avg_in_3f[1], # Corresponds to x1
Avg_InSample_3F_f2 = avg_in_3f[2], # Corresponds to x2
Avg_InSample_3F_f3 = avg_in_3f[3], # Corresponds to x3

Avg_OutOfSample_3F_f1 = avg_out_3f[1], # Corresponds to x1
Avg_OutOfSample_3F_f2 = avg_out_3f[2], # Corresponds to x2
Avg_OutOfSample_3F_f3 = avg_out_3f[3] # Corresponds to x3
))

} # End of outer loop (training_sizes)

# Final Results
print("Simulation Complete. Final averaged results:")
print(results_df)

# Reshape data for ggplot2

# Pivot the data from its wide format (10 metric columns)
# to a long format (3 columns: SampleType, Factor, CVaR_Deviation)
results_long <- results_df %>%
  pivot_longer(
    cols = -TrainingSize,
    names_to = "MetricName",
    values_to = "CVaR_Deviation"
  ) %>%
  # Create new columns based on the metric names
  mutate(
    SampleType = ifelse(grepl("InSample", MetricName), "In-Sample", "Out-of-Sample"),

    # Create descriptive labels for each line
    Factor = case_when(
      MetricName == "Avg_InSample_2F_f1" ~ "2-Factor (x2)",
      MetricName == "Avg_InSample_2F_f2" ~ "2-Factor (x3)",
      MetricName == "Avg_OutOfSample_2F_f1" ~ "2-Factor (x2)",
      MetricName == "Avg_OutOfSample_2F_f2" ~ "2-Factor (x3)",

      MetricName == "Avg_InSample_3F_f1" ~ "3-Factor (x1)",
      MetricName == "Avg_InSample_3F_f2" ~ "3-Factor (x2)",
      MetricName == "Avg_InSample_3F_f3" ~ "3-Factor (x3)",
      MetricName == "Avg_OutOfSample_3F_f1" ~ "3-Factor (x1)",
      MetricName == "Avg_OutOfSample_3F_f2" ~ "3-Factor (x2)",
      MetricName == "Avg_OutOfSample_3F_f3" ~ "3-Factor (x3)",
      TRUE ~ MetricName
    ),

    # Create a simple "Model" column for grouping
    Model = ifelse(grepl("2F", MetricName), "2-Factor", "3-Factor")
  )

# --- 3. Create the plot ---
# This plot shows all 10 lines, grouped by model and sample type
ggplot(results_long,
  aes(x = TrainingSize,
    y = CVaR_Deviation,
    color = Factor, # Color lines by the specific factor
    linetype = SampleType)) + # Dash lines by In/Out-of-Sample
  geom_line(linewidth = 1) +
  geom_point(size = 2) +

```

```

# Use 'facet_wrap' to split the plot into two clean charts:
# One for the 2-Factor model, one for the 3-Factor model.
# This is much easier to read than 10 lines on one chart.
facet_wrap(~ Model, scales = "free_y") +

labs(
  title = "Model Performance vs. Training Set Size",
  subtitle = "Comparing In-Sample (dashed) and Out-of-Sample (solid) CVaR Deviations",
  x = "Number of Rows in Training Set",
  y = "Average CVaR Deviation (0.95)",
  color = "Factor",
  linetype = "Sample Type"
) +
scale_linetype_manual(values = c("In-Sample" = "dashed", "Out-of-Sample" = "solid")) +
theme_minimal() +
# Use a color palette that can handle many lines
scale_color_brewer(palette = "Paired")

```

```

# --- 2. Reshape data for ggplot2 ---
# (Assuming 'results_df' is in your environment from the simulation)

# We "pivot" the data from its wide format (10 metric columns)
# to a long format (several key columns)
results_long <- results_df %>%
  pivot_longer(
    cols = -TrainingSize,
    names_to = "MetricName",
    values_to = "CVaR_Deviation"
  ) %>%
  # Create new helper columns based on the metric names
  mutate(
    SampleType = ifelse(grepl("InSample", MetricName), "In-Sample", "Out-of-Sample"),

    # Create descriptive labels for each line
    Factor = case_when(
      MetricName == "Avg_InSample_2F_f1" ~ "x2 (In-Sample)",
      MetricName == "Avg_InSample_2F_f2" ~ "x3 (In-Sample)",
      MetricName == "Avg_OutOfSample_2F_f1" ~ "x2 (Out-of-Sample)",
      MetricName == "Avg_OutOfSample_2F_f2" ~ "x3 (Out-of-Sample)",

      MetricName == "Avg_InSample_3F_f1" ~ "x1 (In-Sample)",
      MetricName == "Avg_InSample_3F_f2" ~ "x2 (In-Sample)",
      MetricName == "Avg_InSample_3F_f3" ~ "x3 (In-Sample)",
      MetricName == "Avg_OutOfSample_3F_f1" ~ "x1 (Out-of-Sample)",
      MetricName == "Avg_OutOfSample_3F_f2" ~ "x2 (Out-of-Sample)",
      MetricName == "Avg_OutOfSample_3F_f3" ~ "x3 (Out-of-Sample)",
      TRUE ~ MetricName
    ),

    # Create a simple "Model" column for grouping
    Model = ifelse(grepl("2F", MetricName), "2-Factor Model", "3-Factor Model")
  )

```

```

# --- 3. Create the plot ---
# This plot shows all 10 lines, but split by model for clarity
ggplot(results_long,
      aes(x = TrainingSize,
          y = CVaR_Deviation,
          color = Factor,          # Color lines by the specific factor
          linetype = SampleType)) + # Dash lines by In/Out-of-Sample
  geom_line(linewidth = 1) +
  geom_point(size = 1.5) +

# Use 'facet_wrap' to split the plot into two charts
# One for the 2-Factor model, one for the 3-Factor model.
facet_wrap(~ Model, scales = "free_y") +

labs(
  title = "Model Performance vs. Training Set Size",
  subtitle = "Comparing In-Sample (dashed) and Out-of-Sample (solid) CVaR Deviations",
  x = "Number of Rows in Training Set (m)",
  y = "Average CVaR Deviation (Error)",
  color = "Factor & Sample Type",
  linetype = "Sample Type"
) +
scale_linetype_manual(values = c("In-Sample" = "dashed", "Out-of-Sample" = "solid")) +
theme_minimal() +
# Use a color palette that can handle many lines
scale_color_brewer(palette = "Paired")

# --- 1. Calculate Combined OOS Averages ---
# (Assuming 'results_df' is in your environment)

# Create a new data frame for this analysis
analysis_df <- results_df %>%
  mutate(
    # Calculate the average OOS deviation for the 2-factor model
    Avg_OOS_2F_Combined = rowMeans(
      select(., Avg_OutOfSample_2F_f1, Avg_OutOfSample_2F_f2),
      na.rm = TRUE
    ),

    # Calculate the average OOS deviation for the 3-factor model
    Avg_OOS_3F_Combined = rowMeans(
      select(., Avg_OutOfSample_3F_f1, Avg_OutOfSample_3F_f2, Avg_OutOfSample_3F_f3),
      na.rm = TRUE
    )
  )

# --- 2. Find the Break-Even Point ---
# Find all rows where the 3-factor model's combined OOS deviation
# is better (lower) than the 2-factor model's.
break_even_rows <- analysis_df %>%
  filter(Avg_OOS_3F_Combined < Avg_OOS_2F_Combined)

# --- 3. Report the Result ---
if (nrow(break_even_rows) > 0) {
  # Get the *first* training size where this happens
  break_even_point <- min(break_even_rows$TrainingSize)
  print(paste(
    "The break-even point occurs at TrainingSize:",
    break_even_point
  ))
  print(paste(
    "This is the first point where the 3-factor model's average OOS CVaR deviation",
    "becomes lower than the 2-factor model's."
  ))
}

```

```
} else {
  print(paste(
    "The 3-factor model never outperformed the 2-factor model",
    "in the tested training size range."
  ))
}
```

```
# --- 4. (Optional) View the Comparison Table ---
# This helps you see the numbers yourself
print("--- Out-of-Sample Average Comparison ---")
print(select(analysis_df, TrainingSize, Avg_OOS_2F_Combined, Avg_OOS_3F_Combined))
```

```
save.image(file = "problem_CVaR2_err_075.RData")
```